



Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Lista de Exercícios

Código da Aula: 6

Leia as instruções antes de fazer a lista:

Não é necessário testar se os dados passados por argumento são válidos a menos que pedido na questão.

Q1. Escreva uma função chamada `contar_caracteres(texto)` que receba como parâmetro uma string qualquer. A função deve retornar um dicionário no qual as chaves sejam os caracteres individuais da string (convertidos para minúsculas e desconsiderando espaços em branco) e os valores correspondam à quantidade total de ocorrências de cada caractere. Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** "A B C a b a" → **Saída:** {'a': 3, 'b': 2, 'c': 1}
- **Entrada:** "Marinha do Brasil" → **Saída:** {'m': 1, 'a': 3, 'r': 2, 'i': 2, 'n': 1, 'h': 1, 'd': 1, 'o': 1, 'b': 1, 's': 1, 'l': 1}

Resposta Questão 1

```
def contar_caracteres(texto):  
    frequencia = {}  
    texto_limpo = texto.lower().replace(" ", "")  
    for char in texto_limpo:  
        frequencia[char] = frequencia.get(char, 0) + 1  
    return frequencia
```

Q2. Considere um dicionário onde as chaves representam o nome de alunos e os valores representam suas respectivas turmas (exemplo: {"Ana": "Turma A", "Bruno": "Turma B", "Carla": "Turma A"}). Escreva uma função `inverter_dicionario(d)` que receba esse dicionário e retorne um novo dicionário onde as chaves sejam o nome das turmas e os valores sejam listas contendo os nomes dos alunos matriculados em cada uma delas. Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** {"Ana": "Turma A", "Bruno": "Turma B", "Carla": "Turma A", "Daniel": "Turma B"}
- **Saída:** {'Turma A': ['Ana', 'Carla'], 'Turma B': ['Bruno', 'Daniel']}

Resposta Questão 2

```
def inverter_dicionario(d):
```

```

resultado = {}
for aluno, turma in d.items():
    if turma not in resultado:
        resultado[turma] = []
    resultado[turma].append(aluno)
return resultado

```

Q3. Dado o dicionário a seguir, que representa um estoque de produtos e seus respectivos preços:

```

estoque = {
    "camisa": 45.0,
    "calça": 120.0,
    "meia": 12.5,
    "casaco": 250.0,
    "boné": 35.0
}

```

Escreva um trecho de código em Python utilizando laço de repetição e estrutura condicional para construir um novo dicionário denominado `produtos_caros`, contendo apenas os itens que possuem preço estritamente superior a R\$ 50,00. Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** O dicionário `estoque` fornecido.
- **Saída:** `{'calça': 120.0, 'casaco': 250.0}`

Resposta Questão 3

```

estoque = {
    "camisa": 45.0,
    "calça": 120.0,
    "meia": 12.5,
    "casaco": 250.0,
    "boné": 35.0
}

produtos_caros = {}
for produto, preco in estoque.items():
    if preco > 50.0:
        produtos_caros[produto] = preco

print(produtos_caros)

```

Q4. Crie uma função `agrupar_por_tamanho(frase)` que receba uma frase (string). A função deve tratar a frase separando as palavras, ignorando maiúsculas e pontuações simples. O retorno deve ser um dicionário onde:

- As chaves sejam números inteiros representando o comprimento (tamanho) das palavras.
- Os valores sejam listas com as palavras da frase que possuem aquele comprimento exato.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** "O sol brilhou e o sol se foi."
- **Saída:** {1: ['o', 'e', 'o'], 3: ['sol', 'sol', 'foi'], 7: ['brilhou'], 2: ['se']}

Resposta Questão 4

```
import string
```

```
def agrupar_por_tamanho(frase):
    frase_limpa = frase.translate(str.maketrans('', '', string.punctuation)).lower()
    palavras = frase_limpa.split()

    grupos = {}
    for palavra in palavras:
        tamanho = len(palavra)
        if tamanho not in grupos:
            grupos[tamanho] = []
        grupos[tamanho].append(palavra)

    return grupos
```

Q5. Considere a estrutura de dicionário aninhado contendo as notas de estudantes em três disciplinas distintas:

```
alunos = {
    "Ana": {"Matemática": 8.5, "Português": 9.0, "História": 7.5},
    "Bruno": {"Matemática": 5.0, "Português": 6.0, "História": 5.5},
    "Carla": {"Matemática": 9.5, "Português": 8.0, "História": 9.0}
}
```

Escreva um programa que percorra a estrutura e gere um novo dicionário `relatorio_final`. Para cada aluno, calcule a média aritmética das notas e atribua o status `"Aprovado"` se a média for maior ou igual a 7.0, ou `"Recuperação"` caso contrário.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** O dicionário `alunos` fornecido.
- **Saída:** {'Ana': {'media': 8.33, 'status': 'Aprovado'}, 'Bruno': {'media': 5.5, 'status': 'Recuperação'}, 'Carla': {'media': 8.83, 'status': 'Aprovado'}}

Resposta Questão 5

```
alunos = {
    "Ana": {"Matemática": 8.5, "Português": 9.0, "História": 7.5},
    "Bruno": {"Matemática": 5.0, "Português": 6.0, "História": 5.5},
```

```

    "Carla": {"Matemática": 9.5, "Português": 8.0, "História": 9.0}
}

relatorio_final = {}
for aluno, disciplinas in alunos.items():
    notas = disciplinas.values()
    media = round(sum(notas) / len(notas), 2)
    status = "Aprovado" if media >= 7.0 else "Recuperação"
    relatorio_final[aluno] = {"media": media, "status": status}

print(relatorio_final)

```

Q6. Dada uma lista de vendas onde cada item é uma string no formato "PRODUTO:QUANTIDADE":

```
vendas = ["maçã:5", "banana:3", "maçã:2", "uva:1", "laranja:8", "banana:4"]
```

Escreva um algoritmo em Python que processe os dados da lista utilizando divisão de strings (`.split()`) e acúmulo em um dicionário, resultando em uma contagem total consolidada da quantidade vendida de cada produto.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** A lista `vendas` fornecida.
- **Saída:** `{'maçã': 7, 'banana': 7, 'uva': 1, 'laranja': 8}`

Resposta Questão 6

```

vendas = ["maçã:5", "banana:3", "maçã:2", "uva:1", "laranja:8", "banana:4"]

consolidado = {}
for item in vendas:
    produto, quantidade = item.split(":")
    consolidado[produto] = consolidado.get(produto, 0) + int(quantidade)

print(consolidado)

```

Q7. Duas palavras são consideradas anagramas se possuem exatamente as mesmas letras nas mesmas quantidades (ex: "amor" e "roma"). Crie uma função chamada `sao_anagramas(palavra1, palavra2)` que retorne `True` se as palavras forem anagramas e `False` caso contrário.

Restrição: Não utilize métodos ou funções prontas de ordenação (como `sorted()`). A validação deve ser feita obrigatoriamente através da comparação de dois dicionários de contagem de caracteres.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** `sao_anagramas("amor", "roma")` → **Saída:** `True`
- **Entrada:** `sao_anagramas("casa", "saco")` → **Saída:** `False`

```
# Resposta Questão 7
def criar_freq(palavra):
    freq = {}
    for char in palavra.lower():
        freq[char] = freq.get(char, 0) + 1
    return freq

def sao_anagramas(palavra1, palavra2):
    return criar_freq(palavra1) == criar_freq(palavra2)
```

Q8. Em um sistema administrativo, os dados dos usuários são mantidos em uma lista de dicionários conforme o modelo:

```
usuarios = [
    {"nome": "Carlos",
     "email": "carlos@email.com",
     "cpf": "123.456.789-00",
     "ativo": True},
    {"nome": "Mariana",
     "email": "mariana@email.com",
     "cpf": "987.654.321-11",
     "ativo": False},
    {"nome": "Pedro",
     "email": "pedro@email.com",
     "cpf": "456.789.123-22",
     "ativo": True}
]
```

Escreva um código que percorra essa lista e remova a chave `"cpf"` do dicionário apenas dos usuários cujo status `"ativo"` seja `False`, garantindo a privacidade dos dados inativos. Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** A lista `usuarios` fornecida.
- **Saída:**

```
[
    {'nome': 'Carlos',
     'email': 'carlos@email.com',
     'cpf': '123.456.789-00',
     'ativo': True},
    {'nome': 'Mariana',
     'email': 'mariana@email.com',
     'ativo': False},
    {'nome': 'Pedro',
     'email': 'pedro@email.com',
     'cpf': '456.789.123-22',
     'ativo': True}
]
```

Resposta Questão 8

```
usuarios = [
    {"nome": "Carlos", "email": "carlos@email.com", "cpf": "123.456.789-00", "ativo": True},
    {"nome": "Mariana", "email": "mariana@email.com", "cpf": "987.654.321-11", "ativo": True},
    {"nome": "Pedro", "email": "pedro@email.com", "cpf": "456.789.123-22", "ativo": True}
]

for usuario in usuarios:
    if not usuario["ativo"] and "cpf" in usuario:
        del usuario["cpf"]

print(usuarios)
```

Q9. Escreva uma função `mapear_posicoes(texto)` que receba um texto e retorne um dicionário onde:

- As chaves sejam as palavras únicas presentes no texto (em minúsculas).
- Os valores sejam listas de inteiros indicando os índices de posição (iniciando em 0) onde a palavra aparece na sequência das palavras da frase.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- **Entrada:** "o sol brilhou e o sol se foi"
- **Saída:** {'o': [0, 4], 'sol': [1, 5], 'brilhou': [2], 'e': [3], 'se': [6], 'foi': [7]}

Resposta Questão 9

```
def mapear_posicoes(texto):
    palavras = texto.lower().split()
    mapa = {}
    for idx, palavra in enumerate(palavras):
        if palavra not in mapa:
            mapa[palavra] = []
        mapa[palavra].append(idx)
    return mapa
```

Q10. Crie uma função chamada `apurar_votacao(lista_votos)` que receba uma lista de strings contendo os nomes dos candidatos votados em uma eleição (podendo conter nomes repetidos ou votos em branco/nulos). A função deve:

- a) Construir um dicionário com o total de votos recebidos por cada candidato.
- b) Identificar o candidato com o maior número de votos.
- c) Retornar uma tupla contendo o nome do vencedor e o respectivo número de votos no formato `(nome_vencedor, total_votos)`.

Teste a sua função de acordo com as entradas e saídas esperadas a seguir:

- Entrada: ["Alice", "Bob", "Alice", "Nulo", "Bob", "Alice", "Branco"]
- Saída: ('Alice', 3)

Resposta Questão 10

```
def apurar_votacao(lista_votos):  
    contagem = {}  
    for voto in lista_votos:  
        contagem[voto] = contagem.get(voto, 0) + 1  
  
    vencedor = max(contagem, key=contagem.get)  
    return (vencedor, contagem[vencedor])
```